PERGAMON

Control Engineering Practice 7 (1999) 643–653

Hardware-in-the-loop simulation for the design and testing of engine-control systems

R. Isermann*, J. Schaffnit, S. Sinsel

Institute of Automatic Control, Laboratory of Control Engineering and Process Automation, Technical University of Darmstadt, Landgraf-Georg-Str. 4, D-64283 Darmstadt, Germany

Received 18 August 1998; accepted 15 August 1998

Abstract

For the design, implementation and testing of control systems hardware-in-the-loop (HIL) simulation is increasingly being required, where some of the control-loop components are real hardware, and some are simulated. Usually, a process is simulated because it is not available (simultaneous engineering), or because experiments with the real process are too costly or require too much time. The real-time requirements for such simulations depend on the time-scale of the process and the simulated components involved. This paper gives, first, an overview of the various kinds of real-time and HIL simulation. Then, two cases are considered. First, HIL simulation for relatively slow processes, like those in basic industries or heating systems, is discussed. Then, the HIL simulation of combustion engines is shown in detail. The required models are described. Comparisons between real-time simulations and measurements on real diesel engines and trucks are shown. The goal of the HIL system is to develop new control algorithms and to investigate the effect of faults, both in sensors and actuators, and the engine itself. © *1999 Elsevier Science Ltd. All rights reserved.*

Keywords: Real-time simulation; Hardware-in-the-loop simulation; Diesel engines; Turbochargers; Vehicle dynamics; Transputers; Digital signal processors; Simulator testbenches

1. Introduction

The development of many modern products and processes is characterized by the integration of digital control systems. The integration may be performed either by the components (integration of hardware) and/or by the functions (integration of software). Because of the increasing complexity and the inter- relationship between the design of the processes and the design of the control system, computer-aided methods for modeling, simulation and also the design itself, are increasingly being required. This development is also influenced by the decreasing development times (time to market) and the increasing quality, reliability and safety requirements. Typical examples are the design of *mechatronic systems* (e.g., precision mechanic devices, machines and vehicles) (Isermann, 1997).

With regard to the speed of the computation required, *simulation methods* can be subdivided into

- *simulations without* (*hard*) *time limitations*;
- *real-time simulations*;
- *simulations that are faster than real-time.*

Some application examples are given in Fig. 1. Here, *real-time simulation* means that the simulation of a component is performed such that the input and output signals show the same time-dependent values as the real, dynamically operating component. This becomes a computational problem for processes that have fast dynamics compared to the required algorithms and calculation speed.

Different kinds of real-time simulation methods are shown in Fig. 2. The reason for the real-time require ment is mostly that one part of the investigated system is not simulated, but real. Three cases can be distinguished:

(1) The *real process* can be operated together with the *simulated control* by using hardware other than

*Coresponding author. Tel. +49 6151 16 2114; fax: +49 6151 29 3445; e-mail: risermann@iat.tu-darmstadt.de

0967-0661/99/$ - see front matter © 1999 Elsevier Science Ltd. All rights reserved.
PII: S0967-0661(98)00205-6

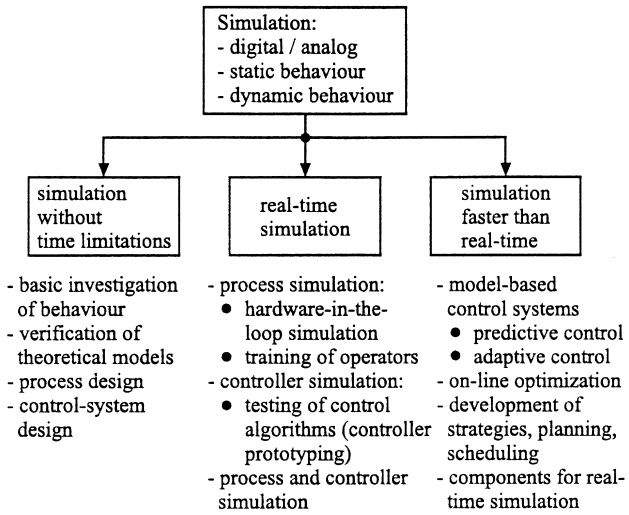

Fig. 1. Classification of simulation methods with regard to the speed and application examples.

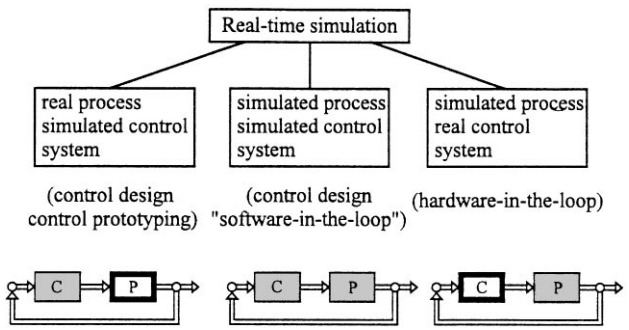

Fig. 2. Classification of real-time simulation.

the final hardware. This is also called "control prototyping".

(2) The *simulated process* can be operated with the *real control hardware*: this is called "hardware-in-the-loop simulation".

(3) The *simulated process* is run with the *simulated control* in real time. This may be required if the final hardware is not available, or if a design step before the hardware-in-the-loop simulation is considered.

In the discussion that follows *hardware-in-the-loop simulation* is considered.

2. Hardware-in-the-loop simulation

2.1. Architectures

Hardware-in-the-loop simulation (HIL) is characterized by the operation of real components in connection with real-time simulated components. Usually, the control-system hardware and software is the real system, as used for series production. The controlled process, consisting of actuators, physical processes and sensors, can then be either fully or partially simulated, compare Fig. 3. Table 1 shows some possible combinations. In general, mixtures of the cases shown are realized. Frequently, some actuators are real, and the process and the sensors are simulated. The reason is that actuators and the control hardware very often form one integrated subsystem, also actuators are difficult to model precisely and to simulate in real time. (The use of real sensors together with a simulated process may require considerable realization efforts, because no physical sensor input exists and it must be generated artificially). In order to change or redesign some functions of the control hardware or software, a bypass unit can be connected to the basic control hardware. Hence, hardware-in-the-loop simulators may also contain partially simulated (emulated) control functions.

The advantages of the hardware-in-the-loop simulation are generally:

- Design and testing of the control hardware and software without the need to operate a real process ("moving the process field into the laboratory");
- Testing of the control hardware and software under extreme environmental conditions in the laboratory (e.g., high/low temperatures, high accelerations and mechanical shocks, aggressive media, electro-magnetic compatibility);

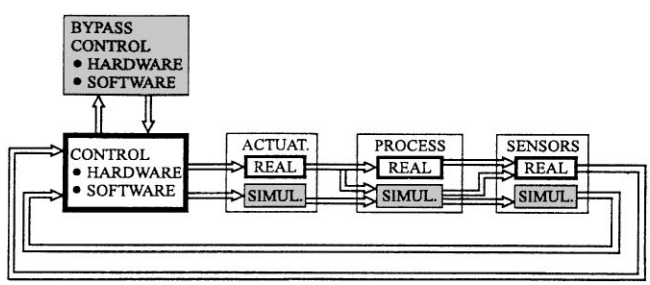

Fig. 3. Hardware-in-the-loop simulation: possible hybrid structures.

Table 1
Hybrid real and simulated process components for hardware-in-the-loop simulation

Cases	Actuators		Process		Sensors	
	Real	Simul.	Real	Simul.	Real	Simul.
1	x	—	—	x	—	x
2	x	—	—	x	x	—
3	x	—	x	—	x	—
4	—	x	—	x	—	x
5	—	x	—	x	x	—

- Testing of the effects of faults and failures of actuators, sensors and computers on the overall system;
- Operating and testing of extreme and dangerous operating conditions;
- Reproducible experiments, frequently repeatable;
- Easy operation with different man-machine interfaces (e.g., in cockpit-design and the training of operators);
- Saving of cost and development time.

2.2. Historical development

The first approaches to HIL simulation were probably realized for (real-time) *flight simulation*, where the early goals were to simulate the instruments with a fixed-cockpit "linktrainer" (1936), (N.N., 1964; Marienfeld, 1965), and later on in addition to move a cockpit according to aircraft motions, e.g. for the training of pilots. Here, the cockpit and the pilot were real, and the motions were generated by electrical and hydraulic actuators. First generations used analog tube controllers and analog motion simulations, which were subsequently replaced by analog and then process computers (N.N., 1953; Anderson, 1962).

HIL motion simulators were also built for the dynamic testing of *vehicle components* (e.g., suspensions, bodies) with hydraulic or electrical actuators (testing machines). Here, for example, the excitation of wheels by a road surface is simulated. Another interesting type of HIL motion simulation is the *vehicle driving simulator* (Drosdol and Panik, 1985; Suetomi et al., 1991). *Dynamic motor teststands*, where the engines are real and the vehicles and gears are simulated by some other hardware (an electrical DC or AC motor) together with a digital process computer, is a special kind of HIL simulator (Thun, 1987; Pfeiffer, 1997).

With the development of digital electronic control systems for vehicles, for example, ABS (antiblock braking systems) for brakes, and ASR (TCS: traction control system) for drive chains and automatic gears, the HIL simulators followed through various stages of development (Kempf et al., 1987; Huber et al., 1988). First versions used high-performance workstations and process computers. However, the amount of real-time simulation was very limited (Klinker, 1992; Wagner and Furry, 1992). The availability of *parallel computers*, in the form of transputers (RISC processors with on-chip RAM and high-speed communication links) and digital signal processors (DSP: microprocessors with efficient operation of additions and multiplications by parallel processing with different memories and buses for the program and the data) then opened the way for real-time simulations of complete hydraulic systems, sensors, actuators and suspension systems, see for instance (Fennel et al. 1992; Hanselmann, 1993). Further research then showed how more comprehensive mechanical systems can be simulated in real time by parallel computers; examples include *multibody systems* (Schaefer, 1993; Rieger and Schiehlen, 1995; Kortuem et al., 1992), *brake systems* (Sailer and Essers, 1996; Sailer, 1996) and *combustion engines* (Spindler and Doll 1990; Savaglio, 1993; Woermann, 1994; Kimura, and Maeda, 1996; Isermann et al., 1997).

Some of these HIL simulations were in-house developments within companies, especially in the fields of aircraft and automobiles, and were published only partially or not at all. Typical configurations consist of either transputer or DSP cards, a host PC and special interface cards for actuators and sensors. Off-the- shelf software exists for the basic operating system and some special processes, sensors and actuators. Commercially available HIL simulators are, for example, described in (Hanselmann, 1993; ETAS, 1997).

2.3. HIL simulation for slow and fast processes

Until now HIL simulation with real controller hardware has mostly been realized for relatively *fast processes* like aircraft, automobiles or combustion engines and their components. The sampling time of the controller is about $T_{0c} \approx 1-10$ ms (sampling frequency $f_{0c} \approx$ 100 Hz–1 kHz), and the sampling time for the simulation of the mechanical processes $T_{0p} \approx 0.5-10$ ms.

The limits on the real-time simulation of these relatively fast processes are given by

- the possibility of using parallel algorithms;
- the kind of integration method used;
- the stiffness of the simulated differential equation systems (expressed as a ratio: largest/smallest eigenvalue).

HIL simulators can also be used to develop and test real controller hardware and software for *slow processes*, i.e. processes in the power, chemical and basic industries. The advantages of "moving the process field into the laboratory" mentioned in Section 2.1 are the same. However, in order to save observation and development time, the simulated process should be run faster than in real time, i.e. to simulate one month of operating time in one day, one hour or even some minutes. Then, the limits are usually not given by the speed of the process simulation, but by the possible speed-up of the real controller hardware. Examples for these limits affect

- speed-up of timers, capture–compare units and watchdog timers;
- speed-up of process interfaces (ADC, DAC);
- speed-up of real actuators;
- time constants of analog components.

However, even speed-up factors of 5 or thereabouts may save considerable time in testing and debugging, compared to field tests. By such HIL simulation for slow processes, not only can development time be saved, but also commissioning time at the plant.

In the following section, the HIL simulation of a very fast process is shown, the dynamic simulation of Diesel engines with real-time simulation of each cylinder working cycle, drive chain and vehicle.

3. Hil simulation of diesel engines with turbocharger

3.1. Process models for real-time simulations of combustion engines

For real-time simulation, dynamic models for the subsystems comprising the diesel engine, the turbocharger and the longitudinal vehicle dynamics are required, see Fig. 4. Several compromises have to be made with regard to the model complexity and the calculation time. For an implementation on a multiprocessor system, a parallel structure of the overall model is aimed at. To ensure the user's acceptance of the simulator, adaptation to the real system must be possible with reasonable effort. There are two main ways of developing mathematical process models:

Theoretical modeling is based on physical laws, expressed by equations. After simplifying assumptions, these equations are stated for single process elements. They can be subdivided into balance equations for the storage of mass, energy and momentum, constitutive equations for sources, transformers and converters, and phenomenological equations for irreversible processes like dissipative elements or sinks. The interconnections of the process elements are described by continuity equations (node law) and compatibility equations (closed circuit law). Based on these equations, an overall model can be calculated. For lumped-parameter systems, the process model can be represented in either state-space form or input/output form, i.e. differential equations or transfer functions. For distributed-parameter systems, in general partial differential equations are obtained. The development of theoretical models is partially supported by domain-specific software tools (e.g. ADAMS, SPICE). First object-oriented languages for multidisciplinary modeling are Omola (Andersson, 1990) Dymola (Elmqvist, 1978) and Modelica (N.N., 1997).

The second method is *experimental modeling* or *identification*. Here, input and output signals are measured, and for a selected class of models, the model is adapted to the process behaviour by minimization of an error measure between the process and the model. For *linear models*, identification methods have reached a very mature state; for instance, methods of Fourier analysis, correlation, and parameter estimation. For *nonlinear models*, different approaches have been formulated. Classical methods are mostly based on polynomial approximators, e.g. Hammerstein, Wiener or nonlinear difference equation (NDE) models. Neural networks are especially attractive for the general task of nonlinear dynamic system identification due to their universal approximation capabilities.

The advantage of system identification is that less time is consumed in the development process. However, the quality of the resulting models is highly dependent on the use of appropriate model structure assumptions, and on the measurement data used. Theoretical models often have a high complexity because of their physical orientation. However, there is a possibility of model reduction, e.g. by simplifying to input/output models with shorter calculation times.

Sometimes the structure of the mathematical model can be determined in an analytical way, but several of the parameters are unknown. In this case, combination with a parameter-identification technique, e.g. parameter estimation, is convenient.

For given equations several simulation software tools already exist, like MATLAB/SIMULINK, ACSL or MATRIX$_x$. For the development of the engine simulator, a combined theoretical–experimental route was taken. This will be illustrated by presenting the different submodels in the following subsection.

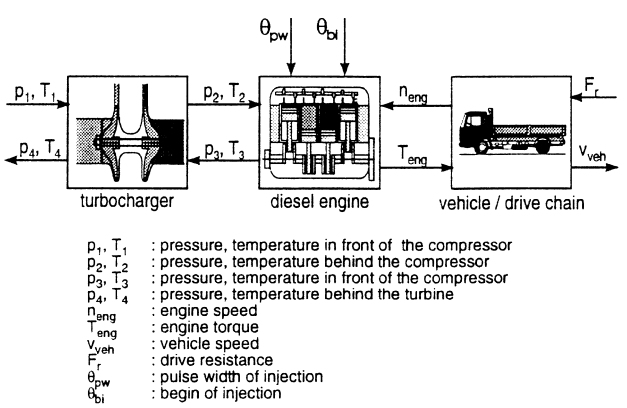

Fig. 4. Simulated process divided into submodels for the diesel engine, turbocharger and vehicle.

3.2. Diesel engine model

In analogy to the real combustion engine, the torque at the crankshaft is determined in the model by superimposing the single dynamic cylinder torques. For the calculation of these cylinder torques, a special torque model was developed, with regard given to the following aims:

- restricted model complexity;
- exact calculation of the average cylinder torque over one working cycle;
- good reproduction of the oscillating cylinder torque during the working cycle;
- easy adaptation of the model to the real engine;
- inclusion of determined or measured multidimensional engine maps.

Calculation of a complete thermodynamic cycle is not feasible, due to the calculation time restrictions and the big effort required for the adaptation. On the other hand, a simple static model based on a simple engine operating map (average torque) is not sufficient for dynamic investigations. The presented torque model considers a compromise between both variants, see Fig. 5.

The approach is based on the idea of decoupling the calculation of the average torque and the dynamic indicated torque over one working cycle:

First, the average cylinder torque $\overline{T_{c,ind}}$ over one working cycle is determined using a multidimensional map. The map can be generated either by processing measured data (e.g., on an engine test stand) or by using suitable software tools (in general, the calculation of a thermodynamic cycle (Pischinger, 1989; Zapf, 1970)). Instead of look-up tables, neural networks can be used. They require fewer parameters, and can be adapted if new data become available.

Below the dynamic indicated cylinder torque $T_{c,ind}(\theta_{CS})$ for one working cycle in dependence on the crank angle θ_{CS} is determined, based on a given average engine torque. It consists of a zero mean *basic torque* $T_{c,basic}(\theta_{CS})$, describing the mass and gas forces, and an *effective torque* $T_{c,eff}(\theta_{CS})$, describing the gas forces resulting from combustion. The mean value of the effective part is equivalent to the average cylinder torque $\overline{T_{c,ind}}$. This approach ensures that the mean value of the dynamically indicated cylinder torque $T_{c,ind}(\theta_{CS})$ is determined correctly, even if the modeling of the dynamics is only approximate. The dynamic cylinder torque $T_c(\theta_{CS})$ at the crankshaft is given by the difference between $T_{c,ind}(\theta_{CS})$ and the cylinder friction torque $T_{c,fric}$.

Basic torque. The basic torque $T_{c,basic}(\theta_{CS})$ is determined using a simplified combustion chamber model. First, the *basic cylinder pressure* $p_{c,basic}(\theta_{CS})$ is calculated, which is an approximation of the cylinder pressure for the non-fired engine. During the compression and combustion cycle, $p_{c,basic}(\theta_{CS})$ follows a polytropic change of state. During the intake and exhaust cycle, $p_{c,basic}(\theta_{CS})$ is equivalent to the pressure at the beginning of compression $p_{c,0}$:

$$p_{c,basic}(\theta_{CS}) = p_{c,0},$$

$$\forall \theta_{CS} \in [0°, 180°] \vee [540°, 720°],$$

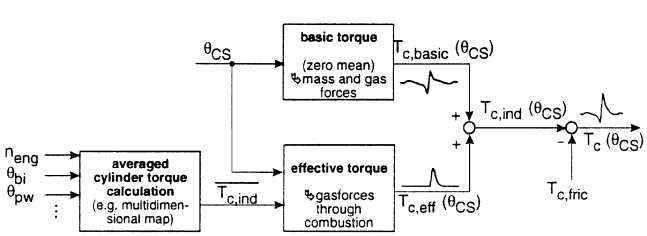

Fig. 5. Cylinder torque generation for real-time simulation.

$$p_{c,basic}(\theta_{CS}) = p_{c,0}\left(\frac{V_{c,0}}{V_c(\theta_{CS})}\right)^n,$$

$$\forall \theta_{CS} \in [180°, 540°]. \quad (1)$$

$V_c(\theta_{CS})$ denotes the current cylinder volume, $V_{z,0}$ the cylinder volume at the beginning of compression, and n the polytropic coefficient. Fig. 6 represents the typical basic pressure for one working cycle. For comparison, the measured cylinder pressure $p_{c,ind}(\theta_{CS})$ for the fired operation is shown. The difference between the two curves is the part that is used for generating the effective torque. The pressure at the beginning of compression $p_{c,0}$ essentially depends on the charge–air pressure, and can be determined by a simplified calculation of the charge cycle (Zapf, 1970).

The basic pressure is converted to the basic torque $T_{c,basic}(\theta_{CS})$ by using the mechanical laws of the crankgear (Sinsel et al., 1997; Maass and Klier, 1981). Here, the resulting gas forces and the oscillating mass forces are taken into consideration.

Effective torque. The effective torque $T_{c,eff}(\theta_{CS})$ is given by the difference between the indicated cylinder torque $T_{c,ind}(\theta_{CS})$ and the zero mean basic torque $T_{c,basic}(\theta_{CS})$:

$$T_{c,eff}(\theta_{CS}) = T_{c,ind}(\theta_{CS}) - T_{c,basic}(\theta_{CS}). \quad (2)$$

This part of the torque essentially results from the combustion during the expansion cycle. Fig. 7 shows the subdivision of the indicated torque into basic and effective torques. The represented indicated torque is based on measured cylinder pressure data acquired on a truck engine (Mercedes-Benz OM 402). $T_{c,eff}(\theta_{CS})$ can be approximated by the following equation:

$$\hat{T}_{c,eff}(\theta_{CS}) = a\theta'^2_{CS}\, e^{(-b\cdot\theta'_{CS})}$$

$$\text{with } \theta'_{CS} = \theta_{CS} - 360°. \quad (3)$$

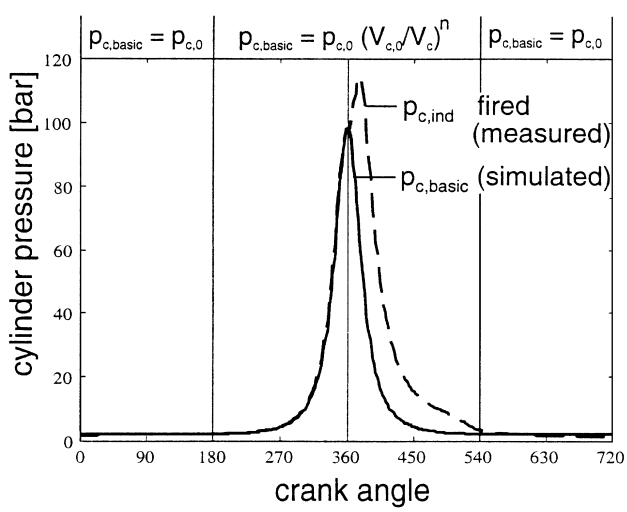

Fig. 6. Comparison between $p_{c,basic}(\theta_{CS})$ and the measured cylinder pressure $p_{c,ind}(\theta_{CS})$ for fired operation.

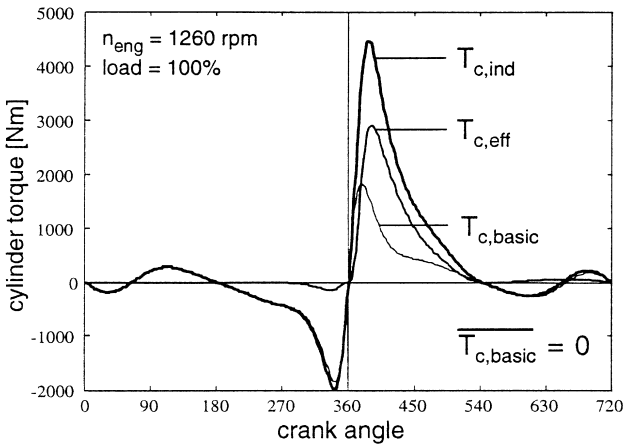

Fig. 7. Cylinder torque, subdivided into basic and effective torque.

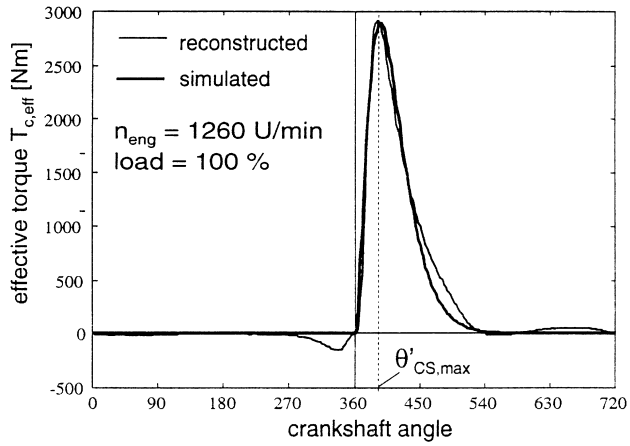

Fig. 8. Effective cylinder torque: simulated, and derived from measurements.

In Eq. (3) it is assumed that the effective torque is equal to zero, except in the combustion cycle. The parameters a und b have to satisfy the following conditions:

1. Because $T_{c,basic}(\theta_{CS})$ is zero mean, the average of the approximated effective torque must be equivalent to the average of the indicated cylinder torque:

$$\frac{1}{720^\circ}\int_0^{720^\circ} \hat{T}_{c,eff}(\theta_{CS})\,\mathrm{d}\theta_{CS} = \overline{T_{c,ind}}. \tag{4}$$

2. The maximum of the approximating function is at crank angle $\theta'_{CS} = \theta'_{CS,max}$. While the crank angle for maximum pressure is influenced by the engine load, engine speed and start of injection, the crank angle for maximum torque is nearly independent of these quantities. This fact is due to the mechanics of the crankgear. Thus $\theta'_{CS,max}$ can be assumed as a constant.

$$\frac{\mathrm{d}}{\mathrm{d}\theta'_{CS}}\hat{T}_{c,eff}\big|_{\theta'_{CS}=\theta'_{CS,max}} = 0. \tag{5}$$

Using Eqs. (4) and (5), the parameters a und b can be obtained:

$$a = \frac{4\,720^\circ\overline{T_{c,ind}}}{(\theta'_{CS,max})^3} \qquad b = \frac{2}{\theta'_{CS,max}}. \tag{6}$$

Using Eqs. (3) and (6), $T_{c,eff}(\theta_{CS})$ can be calculated, depending on $\overline{T_{c,ind}}$. Fig. 8 shows the effective torque simulated and derived from measurements of the cylinder pressure. Using this approach, the dynamics of the overall cylinder torque $T_{c,ind}(\theta_{CS})$ can be reproduced with sufficient accuracy. In Fig. 9, the simulated and reconstructed cylinder torque for one working cycle is depicted. This good agreement can also be obtained at other speed/torque operating points. Only at low load do small deviations occur, due to the simplified model assumptions mentioned above.

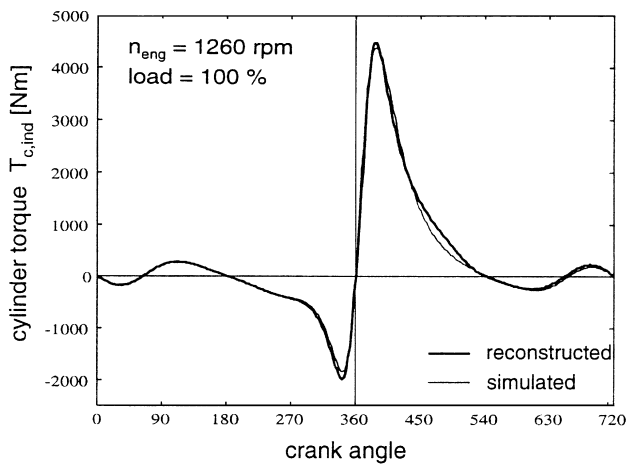

Fig. 9. Indicated cylinder torque: simulated, and derived from measurements.

3.3. Turbocharger model

Fig. 10 schematically represents the exhaust turbocharger with waste gate. The charging process has a nonlinear static input/output behavior, as well as a strong dependency at the dynamics at the operating point.

In general, the static behavior of the turbocharger may be sufficiently described by characteristic maps (look-up tables) of the compressor and turbine. However, if the dynamics of the turbocharger need to be considered, basic mechanical and thermodynamical modeling is required, see (Zinner, 1985; Pucher, 1985).

Practical applications have shown that these methods are capable of reproducing the characteristic dynamic behavior of the turbocharger. The model quality, however, essentially depends on accurate knowledge of several process parameters, which have to be laboriously derived or estimated, in most cases by analogy. A further

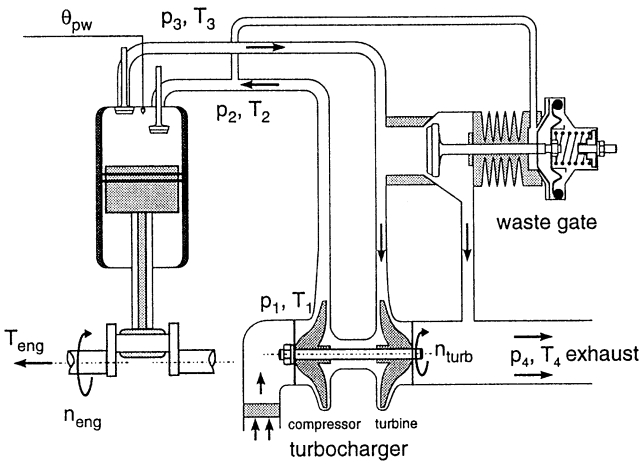

Fig. 10. Diesel engine with exhaust turbocharger and waste gate.

disadvantage is the considerable computational effort due to the complexity of those methods.

For the present application, a simple, easy-to-identify input/output model, suitable for real-time simulation, is required. Therefore, a novel approach to the development of a dynamic model of an exhaust turbocharger was taken, using an artificial neural network called LOLIMOT (Nelles et al., 1996; Nelles and Isermann, 1996). This is a special kind of basis function network, complying with the required properties to a high degree. Only recorded data of input/output measurements of the real process are required. Thus, no theoretical knowledge of the process is necessary. For a hardware-in-the-loop simulation, the pulse width θ_{pw} and the engine speed n_{eng} are chosen as inputs, while the charge–air pressure p_2 is the output. The sampling time is 0.2 s. It was found by trial and error that the turbocharger can be described sufficiently well by assuming a second-order process. Therefore, the charge–air pressure $p_2(k)$ at time instant k is modeled by the following equation:

$$p_2(k) = f(\theta_{pw}(k), \theta_{pw}(k-1), n_{eng}(k), n_{eng}(k-1), p_2(k-1), p_2(k-2)). \quad (7)$$

The training data were generated by a special driving cycle, to excite the system with all the amplitudes and frequencies of interest. Therefore, it covers the input space of the function $f(\circ)$ well. The measurements were recorded on a flat test track. Also, in order to operate the engine in high load ranges, the truck was driven with the largest feasible load. For validation, special driving cycles were recorded, which reproduce realistic conditions in urban and interstate traffic. LOLIMOT was trained until no substantial improvement could be reached by further increasing the model's complexity.

All local linear models have one slow and one fast stable real pole. The slow poles range from 0.71 to 0.84 (representing time constants between 0.6 and 1.2 s), and the fast poles range from 0.12 to 0.37 in the z-domain. This is consistent with the physical knowledge about the turbocharger. The highest gain in all the local linear models is five times larger than the lowest gain. This underlines the strongly non-linear static and dynamic process characteristics.

Fig. 11 shows the validation performance. The data were measured by driving the truck in urban traffic. The maximum output deviation between the simulated and measured charge-air pressures was less than 0.1 bar. The identification of the turbocharger with LOLIMOT as an universal approximator required less than a minute on a Pentium PC. The model development time was only one day, including the trial-and-error approach for dynamic order determination (excluding the time for measurements).

3.4. Vehicle model

To operate the engine model with a realistic load, a suitable model of the vehicle's longitudinal dynamics is required (Pfeiffer, 1997; Germann, 1996; Isermann et al., 1996). The model includes the drive chain and the vehicle body (Fig. 12). Because it is not necessary to simulate the drive chain in full detail for this application, a two-mass-oscillator model is implemented. Simple models for the clutch, gearbox, brake and hydraulic retarder are integrated. The dynamic behavior of the drive chain can be adapted to the real vehicle, either by parametrization of the elasticity and the damping, or by specifying the dominant natural frequency and the relaxation time, respectively. The driving resistance consists of rolling resistance, climbing resistance and aerodynamic drag.

3.5. HIL simulator

In this section, a simulator test bench for the investigation of the engine control system of the new

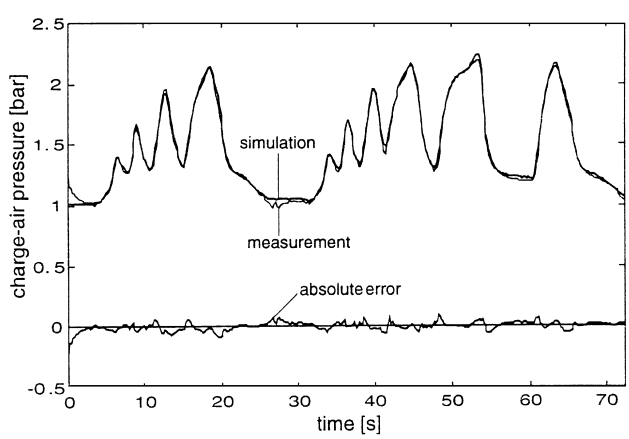

Fig . 11. Comparison between simulated (dynamic neural network) and measured charge-air pressure.

Mercedes-Benz truck engine series 500 and 900 is presented. First, a short survey of the engine management is given.

Electronic engine management. The electronic engine management is part of the *I*ntegrated *E*lectronic *S*ystem (IES), and consists of two control units connected by a CAN-bus interface, Fig. 13.

The *vehicle-engine-control unit* (FMR) contains all the functions concerning the vehicle, e.g.:

- detection of the accelerator-pedal position;
- engine brake control;
- cruise control;
- reduction of the engine torque in conjunction with anti-slip control;
- output of a reference value for the engine speed, for speed control of accessories.

The FMR control unit calculates the reference values for the engine torque, brake torque and engine speed, depending on the driver's commands, the sensor signals and the information received from other units of the IES. The data is transmitted to the *pump-line-nozzle control unit* (PLD) via the CAN-bus. The PLD control unit contains the engine control functions, especially the precise control of the magnetic injection valves. For that, the optimal beginning and duration (pulse width) of injection for each cylinder, depending on the reference values obtained from the FMR control unit, are calculated. By means of multidimensional maps and different sensor signals (sensors for engine speed, charge-air pressure, and temperatures of fuel, coolant and charge air), an optimal operating condition for fuel consumption, emissions and driving comfort is achieved.

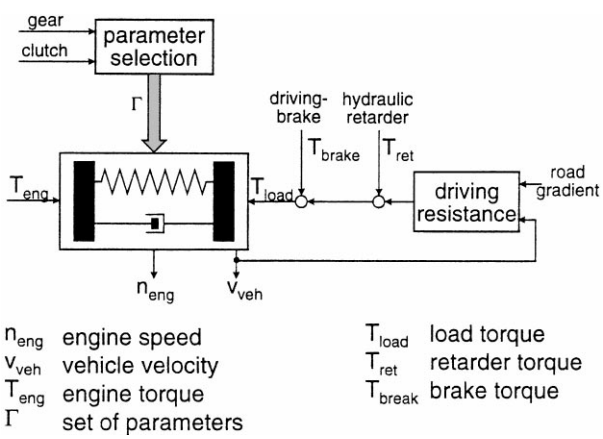

Fig. 12. Vehicle model.

Setup of the simulator test bench. Fig. 14 shows the setup of the simulator test bench for the investigation of the engine control system as mentioned above. It may be subdivided into the following parts:

- real-time computer system, including *I*/*O*-modules;
- interface, consisting of the sensor and actuator interface;
- PLD control unit, including real actuator components, either stand-alone or in combination with the real FMR;
- PC with graphical user interface;
- control panel.

Real-time computer system (*Versions 1 and 2*). To fulfill the real-time constraints necessary for an HIL simulation, powerful computer hardware is required. For the first version of the simulator test bench, a transputer system with 11 processors (T805) was developed. Four transputers were used to calculate the simulation models, and seven for *I*/*O* communication. The models for the different subsystems are calculated in parallel to increase the computing power. The real-time computer system for the second simulator version is based on a dSPACE-system equipped with digital signal processors and

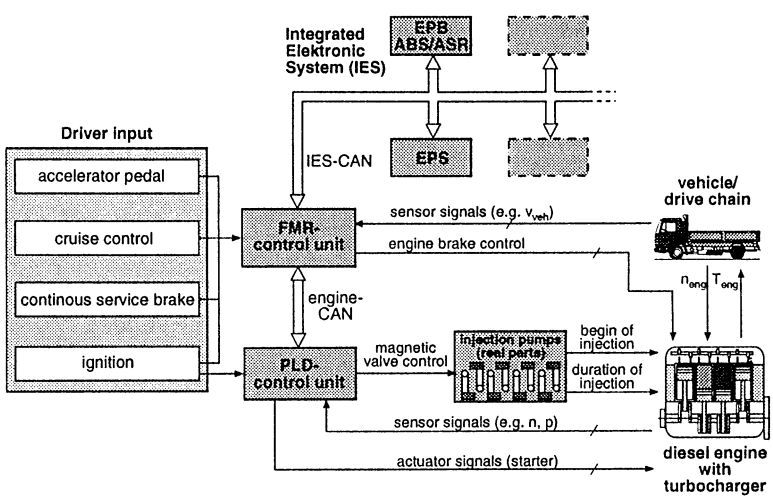

Fig. 13. Engine control system of the new Mercedes-Benz truck engine series 500 and 900.

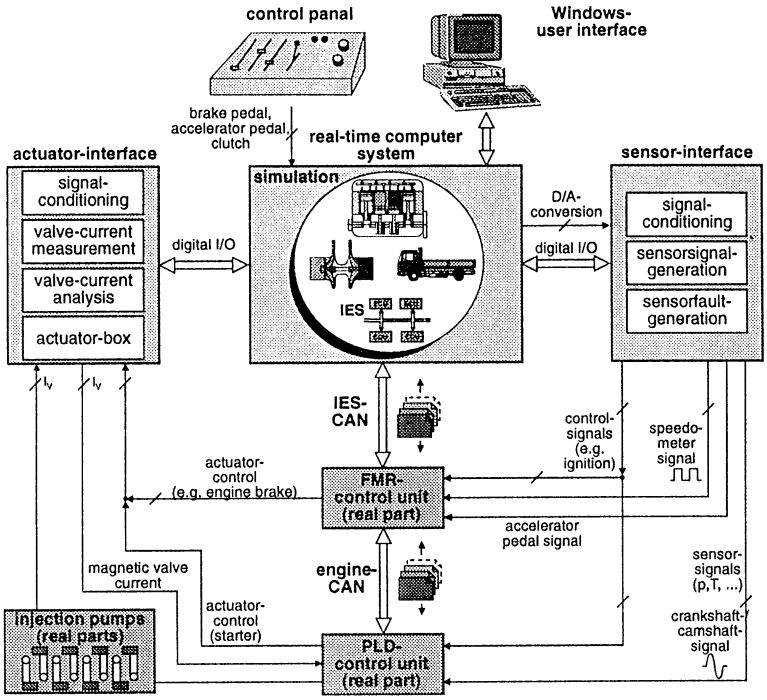

Fig. 14. Setup of the simulator test bench with simulated engine and vehicle, and real engine control unit and injection actuators.

a DEC Alpha processor. This system has the advantage of higher computing power, which makes a parallel calculation unnecessary. It also offers the possibility to realize all the models in MATLAB/SIMULINK, to use all the benefits of a graphical simulation environment. Special I/O-modules (Digital-I/O-module, D/A converter, A/D converter and CAN-interface) are used for the communication with the interface.

Interface. The coupling of the simulator and the control unit is implemented by means of a special interface, which can be subdivided into a sensor and an actuator interface. The *sensor interface* generates the necessary sensor signals, like temperatures and pressures. The pulses of the camshaft and crankshaft inductive speed sensors are generated by a board specifically designed for high-speed signal generation. For that purpose, a look-up table with the pulse signals versus the crankshaft angle is stored off-line in memory. During the simulation, the signals are periodically read out, synchronously with the simulated engine speed. This realization guarantees high flexibility in forming the pulses and adapting different gear wheels. The sensor interface also contains relay electronics to simulate sensor faults like interruptions and short-circuits.

The *actuator interface* consists mainly of the injection pumps (pump-line-nozzle injection system), which are integrated into the simulation test bench as real components, because the combination of the PLD control unit and the injection pumps represents a mechatronic unit that is difficult to model. A special electronic device measures the magnetic valve currents to reconstruct the real valve opening time, and to determine the pulse width and the beginning of injection. These quantities are transferred to the real-time computer system for engine simulation. By this way, the real behaviour of the injection pumps is included.

PLD control unit and FMR control unit as test piece. The simulator test bench was set up with the objective of testing the PLD control unit, either stand alone or in combination with the FMR control unit. In the first case, the necessary FMR functions are simulated by the computer system. The data transfer is done via the engine CAN-bus. In the second case, the PLD and the FMR control units are connected directly via the engine CAN-bus. The computer system emulates other IES units in this operation mode by transmitting the data via the IES-CAN-bus to the FMR.

PC with graphical user interface. For efficient use of the simulator, a comfortable experimental environment is needed. The simulator operation is performed with a Windows user interface on the host PC which copies the functionality of a real truck-cockpit. All the relevant simulation quantities can either be visualized on-line, or recorded for off-line analysis. To ensure reproducible results, a driver simulation is implemented, which can automatically follow a given speed cycle by operating the accelerator, brake, clutch and gears.

Control panel. As an alternative, interactive "driving" of the simulator can be performed manually, using a control panel where the most important cockpit functionalities are realized.

3.6. Simulation results

In the following subsection, three HIL simulation examples for an 8-cylinder truck engine (420 kW) are represented, in order to document the applicability and the performance of the simulator.

Fig. 15 demonstrates the effect of switching off a single injection pump valve. The gearbox is in neutral position, and at the beginning the engine runs at idle speed. The cyclic decrease of the engine torque and the engine speed after the fuel shut-off can be seen directly. The control unit gradually compensates for the missing torque of one cylinder by increasing the pulse width, in order to keep the desired idle speed. A full power acceleration of a 40 t truck including two gear shifts (1) is depicted in Fig. 16. The following effects can be observed:

- oscillations in the drive chain (2);
- limitation of soot (3);
- maximum speed limit regulation (4);
- lagged reaction of the turbo charging pressure (5).

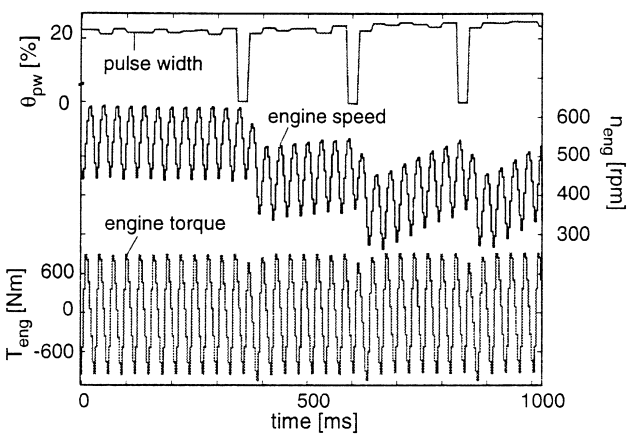

Fig. 15. HIL simulation of switching off a single injection pump valve.

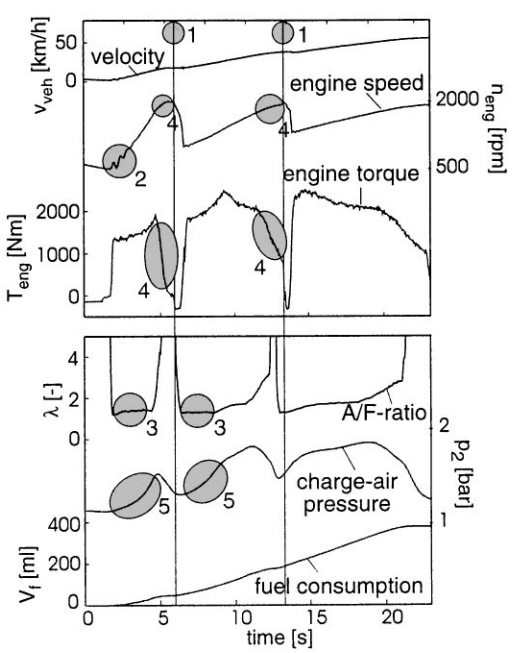

Fig. 16. HIL simulation of a full power acceleration of a 40 t truck.

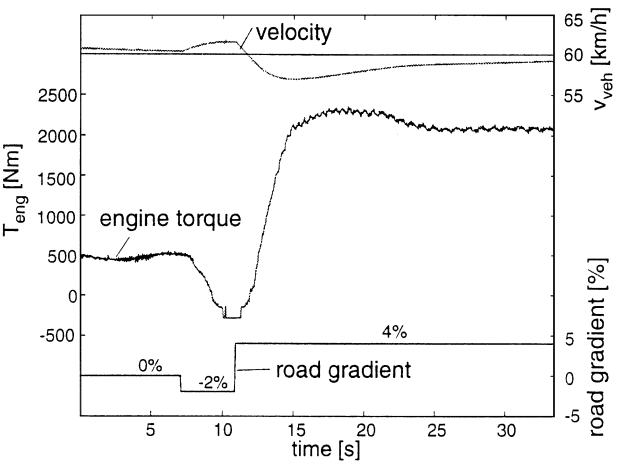

Fig. 17. HIL simulation for the investigation of cruise control.

In Fig. 17 the simulation results of a cruise-control experiment are represented. At the beginning, the 40 t truck runs at a constant speed of 60 km/h on a flat road. One can observe how the cruise control decreases and increases the engine torque as a reaction to changes in the road gradient (first downhill, then uphill) in order to keep to the reference speed.

4. Conclusion

Fulfilling the future constraints on reducing development time and cost will require increased use of real-time simulations. After some general considerations of real-time simulators, architectures and the historical development of hardware-in-the-loop simulation were considered. Several examples during recent years show that hardware-in-the-loop simulation is an efficient tool for the development and testing of engine control systems.

As an application example, a simulator test bench for the investigation of the engine management system of the new Mercedes-Benz truck engine series is described. Several experimental results have demonstrated the possibility of testing engine control functions without the use of a real engine. The HIL-simulator proved to be an efficient tool to develop new control functions, to test the software and hardware of the engine electronics, and to investigate the behaviour in the presence of faults in the sensors, actuators and the engine.

References

Anderson, O.K. (1962). *Design tool (Research and development) simulator survey report*. SAE Nat. Aerospace Engng. Manufacturing Meeting, Los Angeles, Oct. 1962.

Andersson, M. (1990). *Omola – An object-oriented language for model representation*. Lic Tech thesis TFRT-3208, Department of Automatic Control, Lund Institute of Technology, Lund, Sweden.

Drosdol, J., & Panik, F. (1985). The Daimler-Benz driving simulator. SAE 850334.

Elmqvist, H. (1978). *A structured model language for large continuous systems*. Ph.D. dissertation. *Report CODEN*: *LUTFD2/(TFRT-1015)*. Department of Automatic Control, Lund Institute of Technology, Lund, Sweden.

ETAS (1997). *LabCar. Hardware-in-the-loop testsysteme für steuergeräte*. ETAS, Schwieberdingen, Germany.

Fennel, H., Mahr, S., & Schleysing, R. (1992). *Transputer-based real-time simulator – a high performance tool for ABS and TCS development*. SAE 920643.

Germann, S. (1996). *Modellbildung und modellgestützte Regelung der Fahrzeuglängsdynamik*. Dissertation TU Darmstadt. Fortschr.-Ber. VDI Reihe 12 Nr. 309. Düsseldorf, Germany: VDI-Verlag.

Hanselmann, H. (1993). *Hardware-in-the-loop simulation as a standard approach for development, customization, and production test of ECU's*. SAE 931953.

Huber, W. et al. (1988). *Simulation, performance and quality evaluation of ABS and ASR*. SAE 880323.

Isermann, R., Germann, S., Würtenberger, M., Halfmann, C., & Holzmann, H. (1996). Model-based control and supervision of vehicle dynamics. *Fisita Congress*, Prag, Czech Republic, 17–21 June 1996.

Isermann, R. (1997). Mechatronic systems – A challenge for control engineering. *Proc. 1997 American Control Conf.*, Albuquerque, USA, 4–6 June 1997. (Vol. 5, pp. 2617–2632).

Isermann, R., Sinsel, S., & Schaffnit, J. (1997). Hardware-in-the-loop simulation of diesel-engines for the development of engine control systems. *Proc. 4th IFAC-Workshop on Algorithms and Architecture for Real-Time Control, AARTC'97*, Vilamoura, Portugal, 9–11 April 1997 (pp. 90–92).

Kempf, D., Bonderson, L.S., & Slafer, L.I. (1987). *Real time simulation for application to ABS development*. SAE 870336.

Kimura, A., & Meda, I. (1996). Development of engine control system using real time simulator. *Proc. IEEE Int. Symp. Comput. Aid. Contr. Systems Design*, Dearborn, 15–18 Sept. 1996 (pp. 157–163).

Klinker, W. (1992). Hardware-in-the-loop. *SUN News*, (pp. 21–23). October 1992.

Kortuem, W., Sharp, R.S., & Departer, A.D. (1992). *Review of multibody computer codes for vehicle system dynamics*: *Problem requirement codes and experiences with benchmark problems*. Carl Cranz Gesellschaft, Oberpfaffenhofen, Germany.

Maass, H., & Klier, H. (1981). *Kräfte, Momente und deren Ausgleich in der Verbrennungskraftmaschine*. Berlin, Germany: Springer.

Marienfeld, H. (1965). *Flugsimulation*. Dissertation TU Darmstadt.

Nelles, O., & Isermann, R. (1996). Basis function networks for interpolation of local linear models. *Proc. 35th IEEE Conf. on Decision and Control*. Kobe, Japan, Dec. 1996, (Vol. 1, pp. 470–475).

Nelles, O., Sinsel, S., & Isermann, R. (1996). Local basis function networks for identification of a turbo charger. *Proc. UKACC Int. Conf. on Control '96*, Exeter, UK, 2–5 Sept. 1996 (Vol. 1, pp. 7–12).

N.N. (1953). The redifon comet flight simulator. *De haveland gazette*, August 1953, 139–141.

N.N. (1964). Schulung und praxis mit flugsimulator. *INTERAVIA*, *19*(8), 1104–1107.

N.N. (1997). *Modelica, a unified object-oriented language for physical systems modeling. Modelica homepage*: http://www.Dynasim.se/Modelica/

Pfeiffer, K. (1997). *Fahrsimulation eines Kraftfahrzeugs mit einem dynamischen Motorprüfstand*. Dissertation TU-Darmstadt. Fortschr.-Ber. VDI Reihe 12 Nr. 336. VDI-Verlag, Düsseldorf, Germany.

Pischinger, R. (1989). *Thermodynamik der Verbrennungskraftmaschine*. Berlin, Germany: Springer.

Pucher, H. (1985). *Aufladung von Verbrennungsmotoren*. Sindelfingen, Germany: Expert-Verlag.

Rieger, K., & Schiehlen, W. (1995). Echtzeitsimulation eines Fahrzeugmodells mit aktiver Federung. *VDI-Bericht Nr. 1189* (pp. 17–34). Düsseldorf, Germany: VDI-Verlag.

Sailer, U. (1996). *Nutzfahrzeug-Echtzeitsimulation auf Parallelrechnern mit Hardware-in-the-Loop*. Renningen-Malmsheim, Germany: Expert-Verlag.

Sailer, U., & Essers, U. (1996). Parallelverarbeitung als Weg zur Echtzeitsimulation in der Kfz-Steuergeräteentwicklung. *VDI-Bericht Nr. 1283* (pp. 99–116). Düsseldorf, Germany: VDI-Verlag.

Savaglio, C. (1993). Hardware-in-the-loop simulation – an engine controller implementation. SAE 930204.

Schaefer, P. (1993). Echtzeitsimulation aktiver Mehrkörpersysteme auf Transputernetzen. Fortschr.-Ber. VDI Reihe 11 Nr. 202. Düsseldorf, Germany: VDI-Verlag.

Sinsel, S., Schaffnit, J., & Isermann, R. (1997). Hardware-in-the-loop simulation von dieselmotoren für die Entwicklung moderner motormanagementsysteme. Moers, Germany: *VDI-Tagung*.

Spindler, W., & Doll, M. (1990). Ermittlung des stationären und des instationären Betriebsverhaltens von kleinen schnellaufenden Dieselmotoren. *Forschungsberichte Verbrennungskraftmaschinen*: Heft 454, FVV, Gesamtprozeßanalyse-Thermodynamik PKW-Dieselmotoren, Vorhaben 384, Abschlußbericht.

Suetomi, T. et al. (1991). *The driving simulator with large amplitude motion system*. SAE 910113.

Thun, H.-J.v. (1987). Dynamischer Verbrennungsmotorenprüfstand mit Echtzeitsimulation des Kraftfahrzeug-Antriebsstranges. *Automobiltechnische Zeitschrift – ATZ*, *89*(1).

Wagner, J.R., & Furry, J.S. (1992). A real time simulation environment for the verification of automotive electronic controller software. *Int. J. Vehicle Des.*, *13*(4), 365–377.

Woermann, R. (1994). *Ein beitrag zur echtzeitsimulation technischer systeme hoher dynamik mit diskreten modellen*. Dissertation Universität Gesamthochschule Kassel.

Zapf, H. (1970). Untersuchungen zur vorausberech-nung der ladungsendtemperatur in viertakt-die-selmotoren. *Motortechnische Zeitschrift – MTZ*, *12*.

Zinner, K.A. (1985). *Aufladung von verbrennungsmotoren*. Berlin, Germany: Springer.